

CSC 242 — Python Data Structures

CSC 242 Group Programming Assignment: Bracket & Syntax Validator

Unit 5 (Week 6) — Group Project, Option 3 | Total Points: 75 | Group 2 | Group target: Wed Oct 7, 2026 | Due: Sun Oct 11, 2026

This is Group 2's working copy of the assignment directions, written in the same format as our lab directions. The official Directions, Grading Rubric and Group Log Template on D2L are what we are graded on. The rubric in Section "Grading Rubric" below is copied word for word from the official Grading Rubric. The program must run as a single combined program with no errors.

Overview

Our group is building a **Bracket & Syntax Validator**. The program reads a block of text (typed, pasted, or loaded from a file), stores it as a `MyArrayList` of lines, and uses a `MyArrayStack` to check whether every `(`, `[`, and `{` is closed by the matching `)`, `]`, `}` in the correct order. If not, it reports the **line and column of the first problem**. Every check is logged to a `MyLinkedList`-backed session history so the user can list past checks and re-open any one of them.

The three structures depend on each other — the data flows through all three — which is what the rubric's "Correct integration" requirement (1.1, 12 pts) is looking for:

```

TextBuffer          -> check_brackets          -> SessionHistory
MyArrayList of lines  MyArrayStack of open      MyLinkedList of past
(look up line n fast) brackets (nesting = LIFO) checks (grows at the end)
                        |
                        v
                      CheckResult (the shared object all modules agree on)

```

Example of a failed check, as shown in the detail view:

```

Check #1: typed text
  1 | if (x > [1, 2) {
      ^
FAIL - Mismatch at line 1, col 14: expected ']' but found ')'

```

Group Members and Assignments

Member	Files	Branch	Reviewer	Deadline
Andrew Gause agause7975	check_result.py	setup-base-classes	Ramina	Merged Wed 9/30 — everyone depends on it
Andrew Gause	validator_console.py	feature-console-menu	Ramina	PR by Sat 10/3

Member	Files	Branch	Reviewer	Deadline
Benjamin Clark bclark2799	text_buffer.py	feature-text-buffer	Andrew	PR by Sat 10/3
Benjamin Clark	bracket_checker.py	feature-bracket-checker	Andrew	PR by Sat 10/3
Ramina Daood rdaood8933	session_history.py	feature-session-history	Benjamin	PR by Sat 10/3
Ramina Daood	test_end_to_end.py + new files in samples/	test-end-to-end	Benjamin	PR by Sat 10/3

Every file you own has an **ASSIGNED TO** banner at the top, and every method you complete is marked `TODO (YourName)`. For the Session 2 Individual Understanding Check, each member explains the code they **reviewed**: Andrew → Benjamin's checker, Benjamin → Ramina's history, Ramina → Andrew's console.

Required Files

- `my_array_list.py`, `my_linked_list.py`, `my_stack.py`, `my_array_stack.py` — shared base classes (one agreed copy for the whole group), **do not modify**
- `check_result.py` — the shared `CheckResult` interface (Andrew)
- `text_buffer.py` — `TextBuffer`, a `MyArrayList` of lines (Benjamin)
- `bracket_checker.py` — `check_brackets()`, the `MyArrayStack` algorithm (Benjamin)
- `session_history.py` — `SessionHistory`, a `MyLinkedList` of past checks (Ramina)
- `validator_console.py` — the main program and menu (Andrew)
- `test_end_to_end.py` — full-pipeline tests on sample files (Ramina)
- `test_text_buffer.py`, `test_bracket_checker.py`, `test_session_history.py` — provided test drivers, **do not modify**
- `expected_output/` — compare your test output against these files exactly
- `samples/` — sample input files (four given; Ramina adds at least four more)
- `README.md` — project overview for visitors (what it does, how to run it, how the structures fit together)
- `CONTRIBUTING.md` — the team plan: who owns which file, branches, reviewers, schedule, and rules

Given: `CheckResult` Fields and Message Formats

`CheckResult` is the interface agreement between the modules: the checker **creates** it, the history **stores** it, and the console **prints** it. Its four public attributes must be named exactly:

```
is_balanced    True / False
line_number    1-based line of the FIRST problem (0 when balanced)
column         1-based column of the FIRST problem (0 when balanced)
message        text describing the result
```

`str(result)` is `"PASS - " + message` or `"FAIL - " + message`. There are exactly four outcomes, and the messages must match these formats exactly because the tests compare the text:

Outcome	When	Message format
Balanced	Every bracket matched	All brackets are balanced
Mismatch	A closer does not match the most recent opener	Mismatch at line L, col C: expected 'X' but found 'Y'
Unexpected closer	A closer appears when no bracket is open	Unexpected 'Y' at line L, col C: no bracket is open
Unclosed opener	Text ends with an opener still on the stack (report the one on top, where it was opened)	Unclosed 'X' opened at line L, col C

Part 1: `TextBuffer` — MyArrayList of Lines (Benjamin)

- Stores text as a `MyArrayList` of strings, one per line, with no newline characters
- Users see **1-based** line numbers; the `MyArrayList` is 0-based, so line n is stored at index $n - 1$
- `load_text(text)` replaces the contents; an empty string gives 0 lines, and a single trailing `"\n"` does not add an extra empty line
- `load_file(file_name)` returns `True` on success, or `False` with the buffer unchanged if the file cannot be read — never crash
- `add_line`, `get_line` (returns `None` when out of range), `get_line_count`, `get_lines`, and a numbered `__str__`

Part 2: `check\_brackets` — MyArrayStack (Benjamin)

- Scan every character of every line in order; **stop at the first problem**
- Opener: push a tuple `(bracket, line_number, column)` so an unclosed opener can be reported where it was opened
- Closer: if the stack is empty → Unexpected; otherwise read `get_top()`, and if it does not match → Mismatch; if it matches → `pop()`
- After the scan: a non-empty stack → Unclosed (top tuple); otherwise → Balanced with line and column 0
- Why a stack: brackets nest, so the most recently opened bracket must be the first one closed — last-in, first-out

Part 3: `SessionHistory` — MyLinkedList (Ramina)

- Stores each check as a given `HistoryEntry` (check number, label, lines, result), added with `add_last` so the log reads oldest to newest
- Check numbers start at 1; `clear()` empties the log and restarts numbering at 1
- `add_check` saves a **deep copy** of the lines (via `copy_list`) and returns the new entry
- `get_entry(check_number)` walks the list with `get_count() + get_at()`; returns `None` if not found
- `get_summary()` — one row per check; `format_detail(n)` — numbered lines with a ^ under the problem column (failed checks only), then the result

Part 4: `validator\_console.py` — Integration (Andrew)

- One `SessionHistory` for the whole session; a menu loop with six choices: check typed text (finish with `END`), check a file, view history, view a past check in detail, clear history, quit

- `run_check` is the pipeline in one place: `check_brackets → history.add_check → print Check #n: <result>`
- The console contains **no bracket logic and no list logic** — it only calls the other modules' public methods
- Bad input (a missing file, a non-number, an invalid menu choice) prints a message; the program never crashes

Part 5: End-to-End Tests (Ramina)

- `run_file_case` runs the full pipeline on one sample file and prints Expected vs. Actual
- Add at least four new sample files and cases: a balanced file nested 3+ levels deep; an empty file; a longer file (8+ lines) whose only error is on the last line; brackets inside a string such as `print("(")`
- Work out each expected result by hand first; if a test fails, report which module looks wrong — do not edit the expected value to match
- Run it together in Session 2 after every branch is merged

Constraints

- The lines must be stored in `MyArrayList`, open brackets in `MyArrayStack`, and the history in `MyLinkedList` — no Python `list`, `dict`, or `collections.deque` standing in for a required structure (the temporary list from `str.split` in `load_text` is fine)
- Do not modify the base classes, the given blocks (`OPENERS/CLOSERS`, `HistoryEntry`, `MENU`), or the provided test drivers
- Do not rename `CheckResult` fields, method names, or message formats without the whole group agreeing first
- No global variables other than the given constants
- Follow the *Computer Science Coding Conventions at Oakton College Python* document: header block (Author, Date, Purpose, Input, Output), a Precondition/Postcondition docstring on every method, 4-space indentation, and descriptive snake_case names — the program must read like it was written by one person

Required Workflow

- One shared GitHub repository; every task gets its own branch — **never commit directly to main**
- Every pull request is reviewed and approved by at least one other member before merging
- Pull before each work session; push after every working change
- Descriptive commit messages ("Add unclosed-opener check to `check_brackets`", not "fixed stuff"); every member must have commits
- One communication channel (Discord) for all coordination; reply to PR review requests within 24 hours
- **Session 1 — Plan & Divide** (Tue 9/29) and **Session 2 — Integrate & Test** (Mon 10/5): every member logs both sessions
- **Write your own code.** The hints describe what to do, not the code to type — you must be able to explain your part and a teammate's part

Testing Your Program

```
python3 test_text_buffer.py      # Benjamin
python3 test_bracket_checker.py  # Benjamin
python3 test_session_history.py  # Ramina
python3 test_end_to_end.py       # Ramina -- after all branches merge
python3 validator_console.py     # Andrew -- the full program
```

Compare each module driver's output line by line against the file with the same name in `expected_output/`. The module test drivers do not depend on each other — only on `check_result.py` — so each member can test their part as soon as `CheckResult` is merged.

Common Mistakes

- Mixing 0-based indexes and 1-based line/column numbers (off-by-one in the reported position or the ^ marker)
- Expecting `pop()` to return a value — our `pop()` returns nothing; call `get_top()` first, then `pop()`
- Calling `get_top()` on an empty stack instead of checking `is_empty_stack()` first
- Reporting an unclosed opener at the end of the text instead of where it was opened
- Storing a reference to the caller's lines in the history instead of a deep copy
- Creating the `SessionHistory` inside the menu loop, which empties it every time
- Changing a message's wording so it no longer matches the expected output
- Committing to `main`, or merging a pull request nobody reviewed

Step-by-Step Checklist

1. Session 1: confirm the plan, agree on the interface, set up the repo and Discord, hold the Group AI Learning Conversation
2. Andrew: finish `check_result.py` and merge it by Wed 9/30
3. Everyone: pull the updated `main`, create your branch, complete your `TODOS`, and pass your test driver
4. Open pull requests by Sat 10/3; reviewers approve by Sun 10/4
5. Session 2 (Mon 10/5): merge, run `test_end_to_end.py` and the console together, fix bugs on `fix-branches`
6. Write your Session 1 and Session 2 Group Log entries (the day of each session)
7. Draft and review the Specifications Document (Tue 10/6); submit everything (Wed 10/7)

Group AI Learning Conversation (Required, One Per Group)

Before building, the group holds ONE shared AI conversation (all members present, or relayed and reviewed together) to research a design concept the project needs, with real follow-up questions — not a single question-and-done exchange. Suggested topic: *how to design the result object passed between the checker and the history, and why a stack is the right structure for nested brackets*. Submit the transcript to the **Group AI Transcript** D2L folder. This does not replace individual Unit AI Learning Transcripts.

Specifications Document (Required Deliverable)

One document (Word or PDF) for the group, including:

- What the project is: what the program does and which classes it combines, and how
- What was accomplished: the final feature set, and anything planned but not completed
- Each member's individual contribution: what each person wrote, reviewed, or tested — including honest notes on any shortcomings or uneven participation
- Meeting notes: a brief summary of Session 1 and Session 2
- Future potential: what we would build next (for example, ignoring brackets inside strings and comments)
- A link to the group's GitHub repository

Grading Rubric (Official — 75 Points)

Copied word for word from the official Group Programming Assignment Grading Rubric. This rubric is scored ONCE per group for Sections 1-3 (the shared deliverable), and individually per student for Sections 4-6 (the accountability components). A group producing excellent code can still have individual members lose points on Sections 4-6.

Section 1: Integration & Functionality (30 pts) — Group Score

Requirement	Description	Max Points
1.1 Correct integration	The program genuinely combines 2+ required classes (MyArrayList, MyLinkedList, MyStack/MyArrayStack, MyCalc) such that they depend on each other for the program's core function — not merely used side by side	12
1.2 Runs correctly	The combined program runs without errors and behaves correctly end to end	8
1.3 Feature completeness	The chosen project option's core described behavior is fully implemented	6
1.4 Code quality & consistency	Consistent naming, indentation, and commenting across the whole codebase, per the Computer Science Coding Conventions at Oakton College Python document — reads as one program, not four stitched-together pieces	4

Section 2: GitHub & Workflow Evidence (15 pts) — Group Score

Requirement	Description	Max Points
2.1 Feature branch workflow	Evidence of separate branches per task; no direct commits to main for feature work	6
2.2 Pull requests & code review	At least one reviewed, approved pull request per branch before merging	6
2.3 Commit quality	Descriptive commit messages; commit history shows contributions from every member	3

Section 3: Specifications Document (15 pts) — Group Score

Requirement	Description	Max Points
3.1 Project description	Clearly explains what the program does and which classes it combines and how	4
3.2 Individual contributions	Honestly documents each member's specific contribution, including any shortcomings or uneven participation	5
3.3 Meeting notes	Summarizes what happened in Session 1 (Plan & Divide) and Session 2 (Integrate & Test)	3

Requirement	Description	Max Points
3.4 Completeness & future potential	Documents what was accomplished, what wasn't, and what the group would build next	3

Shared Deliverable Subtotal: 60 points

Section 4: Group Log Entries (8 pts) — Individual Score

Requirement	Description	Max Points
4.1 Session 1 log	Individual entry submitted for the Plan & Divide session, following the required format	4
4.2 Session 2 log	Individual entry submitted for the Integrate & Test session, following the required format	4

Section 5: Group AI Learning Conversation (5 pts) — Group Score, Applied to Every Member

Requirement	Description	Max Points
5.1 Conversation quality	One shared transcript researching a genuine design concept the project needed, with real follow-up depth (not a single question-and-done exchange)	5

Section 6: Individual Understanding Check (2 pts) — Individual Score

Requirement	Description	Max Points
6.1 Explains teammate-written code (written)	Pass/Needs Follow-Up/Not Pass, based on the required explanation in the member's Session 2 Group Log entry (see Group Log Template) — per the performance table in the Directions document. Pass = 2 pts, Needs Follow-Up = 1 pt, Not Pass = 0 pts	2

Performance	What It Looks Like
Pass	Can describe, in their own words, what a teammate-written part of the program does and roughly how, even without reciting exact code
Needs Follow-Up	Can describe WHAT the part does but not roughly HOW, or vice versa — instructor may request a brief written clarification before deciding
Not Pass	Cannot describe a teammate-written part of the program at all

Individual Accountability Subtotal: 15 points

TOTAL: 60 (Shared Deliverable) + 15 (Individual Accountability) = 75 points

A Note on Individual Variation (from the official rubric): Sections 1-3 and 5 are scored once and applied to every group member equally — collaboration means shared ownership of the outcome. Sections 4 and 6 are scored per individual, so two members of the same group CAN end up with different final scores if one skips a Group Log entry or cannot pass the Individual Understanding Check. This is intentional: it's how the assignment protects against one or two members carrying the whole project while others coast.

Submitting the Project

- **Andrew** submits the project folder (all source files, running as a single combined program with no errors) and the Specifications Document to the **Group Programming Assignment D2L** folder on behalf of the group

- The GitHub repository link must show feature branches and at least one reviewed pull request per branch
- The Group AI Learning Conversation transcript (one per group) goes to the **Group AI Transcript** D2L folder
- **Every member** submits their own two Group Log files together as ONE submission to the **Group Log** D2L folder, named `Unit5_GroupLog_Session1_J Smith jsmit1234.pdf` and `Unit5_GroupLog_Session2_J Smith jsmit1234.pdf` (or .docx), with full last name and Oakton username in each file's header

See the official Group Programming Assignment Directions, Grading Rubric, and Group Log Template on D2L for the complete requirements.